

SmartLogistics — Intelligent Supply Chain & Delivery Orchestration Platform [Part - A]

TransFleet is building **SmartLogistics**, an intelligent large-scale logistics platform designed to support modern e-commerce businesses, retail chains, warehouses, third-party logistics providers, and enterprise supply chain operations.

The company is experiencing rapid growth in shipment volume, warehouse operations, courier partners, and delivery requests, which has exposed several limitations in their current systems:

- Shipment creation and dispatch workflows are slow and partially manual, delaying order fulfillment.
- Inventory data across warehouses and fulfillment centers is inconsistent.
- Courier assignment and route planning are inefficient during peak delivery windows.
- Shipment tracking events are delayed, duplicated, or lost between systems.
- High operational traffic results in delays when processing large dispatch batches and status updates.
- Logistics interactions generate valuable operational data, but it is not being leveraged for optimization or decision-making.

To address these challenges, TransFleet is commissioning a new backend for SmartLogistics with the following business goals.

Description / Problem Statement

This part focuses on building the **foundational backend system** of SmartLogistics, solving scalability, consistency, and reliability challenges in shipment management, warehouse operations, dispatch orchestration, courier assignment, tracking, and analytics.

The goal is to solve:

- Slow and manual shipment dispatch workflows
- Inventory inconsistency across warehouses and fulfillment centers
- High latency during peak operational traffic
- Lack of reliable background processing for logistics operations
- Weak observability and failure recovery mechanisms

Business Goals & Vision

SmartLogistics must provide:

A Robust Shipment & Warehouse Management System

Operations teams should be able to create shipments, assign warehouses, reserve stock, schedule dispatches, and monitor fulfillment progress.

Warehouse teams should manage stock movement, inbound/outbound inventory, and package readiness.

Couriers should receive assignments, update delivery progress, and complete deliveries through reliable workflows.

A Scalable and Reliable Operations Backbone

Creating or dispatching a shipment may trigger multiple internal processes such as:

- Inventory reservation
- Courier assignment
- Label generation
- Route preparation
- Tracking initialization
- Notification scheduling

Updating shipment status may trigger:

- Customer notifications
- Analytics updates
- Reassignment workflows
- Escalation rules

Consistent and Accurate Logistics Data

The state of inventory, shipments, courier assignments, tracking milestones, delivery attempts, returns, and exceptions must be reliable, durable, and easy to query.

A Foundation for Long-Term Scalability

As SmartLogistics grows, the platform should handle:

- Tens of thousands of daily shipments
- High concurrent status updates
- Peak seasonal delivery spikes

- Multi-city warehouse coordination
- Reliable background workflows under heavy load

Core Functional Requirements

1. Shipment & User Management

- Creation and updating of shipments, packages, delivery addresses, and shipment metadata
- User registration with appropriate roles:
 - customer support
 - warehouse operator
 - courier
 - admin
- Courier profile management and operational availability
- Shipment history and audit trail

Each update should ensure consistency across all dependent systems.

2. Dispatch Workflow

When a shipment is approved for dispatch:

- Shipment details should be validated
- Inventory should be reserved or confirmed
- Shipping labels should be generated
- Courier should be assigned
- Tracking journey should be initialized
- Shipment should be marked as **dispatched** only after successful processing

Partial failures must not corrupt shipment state.

3. Delivery Workflow

When a courier receives and processes a shipment:

- Pickup should be recorded
- Status updates should be tracked
- Delivery attempts should be captured
- Final delivery confirmation should be stored
- Notifications may be sent

- Analytics should be updated

This workflow must support:

- High volume field updates
- Idempotent retries
- Network failure recovery
- Duplicate update prevention
- Real-time status visibility

4. Distributed & Event-Driven Behaviors

Examples include:

- Shipment creation events
- Dispatch completion events
- Courier assignment changes
- Tracking milestone updates
- Delivery completion events
- Return initiation events
- Analytics processing events

These tasks should:

- Run independently from user-facing flows
- Be traceable and recoverable
- Handle failures gracefully
- Avoid double-processing
- Support workload spikes efficiently

5. Analytics Metrics

- Total Shipments
- Shipments Dispatched Over Time
- Delivered Shipments
- Failed Deliveries
- Average Delivery Time
- Courier Utilization Rate
- Warehouse Throughput
- Return Rate
- Failed Events / Workflow Issues

6. System Observability & Reliability Expectations

- Clear separation of responsibilities between services/modules
- Monitoring and logging for all critical flows
- Ability to diagnose failures in:
 - Dispatch workflows
 - Courier assignment
 - Tracking updates
 - Warehouse reservation
 - Background workers
- High consistency across all operational data models

Expected Outcomes

- Support complete shipment lifecycle operations
- Reliable dispatch and delivery workflows
- High scalability under traffic spikes
- Strong consistency and recoverability
- Maintainable and production-grade architecture
- PRD Requirement
 - Key use-cases
 - Functional and non-functional requirements
 - Delivery timeline / milestones
 - Traceability between features and deliverables

Tech Stack

Backend

- Python
- FastAPI
- PostgreSQL
- NoSQL DB
- Redis
- Celery Workers with RabbitMQ
- Kafka + Schema Registry
- Temporal (workflows)

Observability

- Prometheus + Grafana
- Jaeger

- OpenTelemetry

DevOps

- Docker
- Docker Compose